

Mengapa ReLU Dipilih untuk Aktivasi dalam Lapisan Tersembunyi dibandingkan Sigmoid?

Pertanyaan ini merupakan salah satu pertanyaan klasik dalam bidang *Machine Learning* dan *Deep Learning*. Pada awal perkembangan *Neural Network*, fungsi aktivasi **Sigmoid** sangat populer. Namun, ketika arsitektur jaringan menjadi semakin dalam (*Deep Learning* seperti CNN), Sigmoid mulai ditinggalkan untuk *hidden layer* dan digantikan oleh **ReLU (Rectified Linear Unit)**.

Berikut tiga alasan utama mengapa ReLU lebih unggul dibandingkan Sigmoid:

1. Mengatasi Masalah *Vanishing Gradient*

Masalah utama pada Sigmoid adalah terjadinya *Vanishing Gradient* saat proses *Backpropagation*.

Fungsi Sigmoid menggunakan rumus:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Fungsi ini meremas semua nilai menjadi rentang 0 sampai 1. Akibatnya, nilai turunannya sangat kecil (maksimal hanya 0,25). Pada jaringan yang sangat dalam, gradien kecil tersebut terus dikalikan sehingga mendekati nol. Hal ini menyebabkan lapisan awal jaringan berhenti belajar karena tidak mendapatkan sinyal pembaruan.

Sebaliknya, ReLU memiliki fungsi sederhana:

$$f(x) = \max(0, x)$$

Jika input bernilai positif, turunan ReLU bernilai 1. Karena gradien tidak mengecil drastis, sinyal pembelajaran dapat diteruskan hingga lapisan awal jaringan. Inilah alasan mengapa ReLU mampu mempercepat proses pelatihan model *Deep Learning*.

2. Efisiensi Komputasi Lebih Tinggi

Pelatihan CNN melibatkan jutaan parameter dan miliaran operasi matematika.

- **Sigmoid** membutuhkan operasi eksponensial (e^{-x}) yang cukup berat secara komputasi.
- **ReLU** hanya melakukan operasi sederhana:
 - Jika nilai $> 0 \rightarrow$ gunakan nilai tersebut

- Jika nilai $\leq 0 \rightarrow$ ubah menjadi 0

Karena operasinya ringan, ReLU membuat proses *training* jauh lebih cepat dan efisien, terutama saat menggunakan GPU.

3. Menciptakan *Sparse Representation*

ReLU mengubah semua nilai negatif menjadi nol. Akibatnya, tidak semua neuron aktif secara bersamaan.

Kondisi ini disebut *Sparse Representation*, yaitu hanya neuron penting saja yang aktif. Keuntungan dari metode ini:

- Model menjadi lebih efisien
- Lebih mudah mengenali fitur penting
- Lebih tahan terhadap *noise*
- Mengurangi kompleksitas perhitungan

Berbeda dengan Sigmoid yang hampir selalu mengaktifkan neuron karena outputnya selalu berada di antara 0 dan 1.

Kesimpulan

ReLU dipilih dibandingkan Sigmoid karena:

1. Mengatasi masalah *Vanishing Gradient*
 2. Proses komputasi lebih cepat
 3. Membentuk jaringan yang lebih efisien melalui *Sparse Representation*
-

Kekurangan ReLU dan Cara Mengatasinya

Meskipun ReLU sangat populer, fungsi ini memiliki kelemahan utama yang dikenal sebagai:

Dying ReLU Problem

Karena ReLU mengubah semua nilai negatif menjadi nol, sebuah neuron bisa “mati” jika selama proses pelatihan selalu menerima input negatif.

Akibatnya:

- Output neuron selalu 0
- Gradien menjadi 0
- Neuron tidak dapat belajar lagi

Jika terlalu banyak neuron mati, akurasi model akan berhenti meningkat.

Selain itu, ReLU tidak memiliki batas atas (*unbounded*), sehingga output yang terlalu besar dapat menyebabkan *Exploding Gradient*.

Cara Mengatasi Kekurangan ReLU

1. Menggunakan Leaky ReLU

Leaky ReLU memberikan sedikit gradien pada nilai negatif:

$$f(x) = \max(0.01x, x)$$

Dengan demikian, neuron tetap memiliki peluang untuk belajar meskipun input bernilai negatif.

2. Menggunakan Batch Normalization

Batch Normalization menjaga distribusi data tetap stabil di sekitar nol sehingga mengurangi kemungkinan neuron masuk ke zona negatif secara permanen.

Manfaatnya:

- Training lebih stabil
 - Mengurangi *Dying ReLU*
 - Mempercepat konvergensi model
-

3. Menggunakan He Initialization

Metode **He Initialization** dirancang khusus untuk jaringan yang menggunakan ReLU.

Tujuannya adalah:

- Mengatur distribusi bobot awal agar stabil
 - Mengurangi risiko neuron mati sejak awal pelatihan
-

4. Menurunkan *Learning Rate*

Learning Rate yang terlalu besar dapat membuat pembaruan bobot menjadi terlalu agresif.

Solusinya:

- Gunakan nilai kecil seperti 0.001 atau 0.0001
 - Gunakan *Learning Rate Scheduler* agar pelatihan lebih stabil
-

Mengapa *Feature Extraction* Dianggap Otomatis?

Konsep utama CNN adalah kemampuan melakukan *Feature Extraction* secara otomatis tanpa perlu diprogram manual.

1. Pendekatan Tradisional (*Hand-crafted Features*)

Sebelum CNN populer, programmer harus menentukan sendiri fitur penting pada gambar.

Contoh:

- Bentuk telinga
- Garis tepi objek
- Tekstur tertentu

Metode ini sulit dan tidak fleksibel karena sangat bergantung pada aturan buatan manusia.

2. Pendekatan CNN (*Automatic Feature Extraction*)

Pada CNN, manusia hanya menyediakan:

1. Dataset gambar
2. Label kelas gambar

Selanjutnya, model akan belajar sendiri mengenali pola melalui proses *training*.

Cara CNN Belajar Fitur

Filter Awalnya Acak

Filter konvolusi pada awal pelatihan hanya berisi angka acak.

Belajar Melalui Kesalahan (*Backpropagation*)

Saat prediksi salah, model menghitung kesalahan (*loss*) lalu memperbarui filter agar prediksi berikutnya lebih akurat.

Membentuk Hirarki Fitur

Secara otomatis, setiap lapisan CNN memiliki tugas berbeda:

- **Lapisan awal:** mendeteksi garis, warna, dan tepi
- **Lapisan tengah:** mendeteksi bentuk dan tekstur
- **Lapisan akhir:** mengenali objek kompleks seperti wajah, mobil, atau hewan

Kesimpulan

Feature Extraction dianggap otomatis karena model CNN mampu mempelajari sendiri pola penting dari data tanpa perlu aturan manual dari manusia.

Mengatasi Kesalahan Analisis pada Gambar Resolusi Tinggi

Masalah ini sering terjadi ketika gambar beresolusi tinggi dipaksa masuk ke model dengan ukuran input kecil, misalnya:

$32 \times 32 \times 3$

Kondisi ini menyebabkan:

- Detail gambar hilang
- Objek menjadi buram (*blurry*)
- Model mengalami salah prediksi

Fenomena ini disebut:

Information Loss akibat Downsampling

Cara Mengatasinya

1. Menggunakan *Smart Cropping*

Jangan langsung melakukan *resize* seluruh gambar.

Langkah yang benar:

1. Fokuskan area objek terlebih dahulu (*cropping*)
2. Baru lakukan *resize*

Dengan cara ini, objek utama tetap terlihat jelas.

2. Meningkatkan Ukuran Input Model

Gunakan ukuran input yang lebih besar, misalnya:

$224 \times 224 \times 3$

Ukuran ini lebih cocok untuk gambar dunia nyata karena detail visual masih terjaga.

3. Menggunakan Dataset Resolusi Tinggi

Dataset seperti CIFAR-10 hanya memiliki resolusi:

$32 \times 32 \times 3$

Jika ingin memproses gambar HD, gunakan dataset resolusi tinggi seperti:

- ImageNet
 - Dataset Kaggle berkualitas HD
 - Dataset kustom dari kamera sendiri
-

4. Menggunakan *Transfer Learning*

Daripada melatih model dari awal, gunakan model yang sudah dilatih sebelumnya seperti:

- VGG16
- ResNet50

- MobileNetV2

Model-model tersebut sudah terlatih menggunakan jutaan gambar resolusi tinggi sehingga lebih akurat dan stabil.

Kesimpulan Akhir

Kesalahan analisis pada gambar sering terjadi karena resolusi input terlalu kecil sehingga informasi visual penting hilang.

Solusi terbaik meliputi:

- Menggunakan *cropping*
- Memperbesar ukuran input
- Menggunakan dataset berkualitas tinggi
- Memanfaatkan *Transfer Learning*

Dengan langkah tersebut, model CNN akan mampu mengenali objek dengan jauh lebih akurat dan stabil.